

LECTURE 1 – INTRODUCTION

INTRODUCTION TO AUTONOMOUS SYSTEMS · MODULE I

Topics: Autonomy spectrum · Perception–Decision–Action loop · The system tuple · Historical arc

LEARNING OBJECTIVES

By the end of this lecture you will be able to:

1. Define autonomy operationally and place a given system on the SAE spectrum
2. Identify the four stages of the Perception–Decision–Action loop and the signal at each interface
3. State the tuple $(\mathcal{X}, \mathcal{O}, \mathcal{A}, f, h, \pi)$ informally and explain the role of each component
4. Explain why partial observability — not lack of sensing power — is the central challenge in autonomous systems

SO · COURSE PLAN

WHAT THIS COURSE IS AND ISN'T



THE CENTRAL QUESTION

How can we build a machine that autonomously interacts with the physical world?

This course is **not** a complete treatment of the field. A full treatment would require separate courses in:

- Differential equations and dynamical systems
- Linear algebra and matrix analysis
- Probability, Bayesian inference, optimisation
- Control theory, reinforcement learning, modern AI

Goal instead: a first coherent view of the entire landscape — every major component introduced, enough to understand the problems, why they are hard, and where to go next.

This course is an entry point, not an endpoint.

THE UNIFYING TUPLE

Every lecture is organised around one mathematical abstraction:

$$(\mathcal{X}, \mathcal{O}, \mathcal{A}, f, h, \pi)$$

SYMBOL	MEANING	FIRST ADDRESSED
$\mathcal{X}$	State space	Module II
$\mathcal{O}$	Observation space	Module II
$\mathcal{A}$	Action space	Module II
f	Dynamics: $x_{t+1} = f(x_t, a_t)$	Modules II, IV, VI
h	Observation model: $o_t = h(x_t)$	Module III
π	Policy: $a_t = \pi(o_t)$	Modules IV–VI

Each topic in the course is a **refinement of one component** of this tuple.

COURSE MODULES

MODULE	LECTURES	FOCUS
I. What Is an Autonomous System?	1	Autonomy, PDA loop, the tuple — intuition before formalism
II. Mathematical Framework	2–3	Dynamical systems, stability, probability, optimisation
III. Perception & State Estimation	4–5	Observation models (h), belief, Kalman & particle filters
IV. Classical Decision Making	6–8	Planning by search, optimal control (HJB/PMP/LQR), MPC
V. Learning-Based Decision Making	9–11	Imitation learning, RL foundations, policy optimisation
VI. Modern Autonomous Intelligence	12–14	Diffusion policies, world models, foundation models / VLAs
VII. Reliability & Open Questions	15–16	Safety constraints, robustness, what remains unsolved

EVALUATION

COMPONENT	WEIGHT
Assignments (6)	50%
Oral exam	30%
Semester project	20%

Bonus points for additional exercises and active participation — can raise the final grade, never lower it.

Theory track only. Engineering, simulation, and implementation are covered in the parallel practicum course.

Prerequisites: Linear algebra, probability, calculus. Introductory ML helpful but not required.

ASSIGNMENTS (ONE PER MODULE)

#	TITLE	RELEASED
1	Mathematical Foundations	After Lecture 3
2	Perception and State Estimation	After Lecture 5
3	Planning and Optimal Control	After Lecture 8
4	Learning to Act	After Lecture 11
5	Modern Autonomous Intelligence	After Lecture 14
6	Safety and Reliability	After Lecture 15

Mix of **theorem proving**, **algorithm implementation**, and **policy training**. Submitted individually.

SEMESTER PROJECT

Shared with the engineering course. Runs the full semester.

Goal: build a working autonomous system in simulation demonstrating the full PDA loop, designed from first principles.

WEEK	DELIVERABLE
3	Topic proposal: system, tuple specification, chosen algorithms
7	Perception + state estimation working in simulation
11	Planning/control integrated; preliminary demo
15	Full system demo + theory report submitted
16	Short presentation and Q&A

ORAL EXAM

End of semester. Covers all theoretical content from **Modules I–VII**.

Question types:

- Define and explain a concept precisely
- Prove a result from the course or assignments
- Apply the tuple to a novel system described in the question
- Analyse a design choice or failure scenario using course concepts

Students draw two questions and may be asked follow-up questions on either.

S1 · WHAT IS AUTONOMY?

TWO DEFINITIONS AND A SPECTRUM



TWO DEFINITIONS

Process-oriented — what an autonomous system *does*:

Definition. An autonomous system continuously **perceives** its environment, builds an **internal representation**, makes **decisions**, **executes** actions, and **adapts** — while pursuing a goal under uncertainty.

This reads like a blueprint for the whole course: perceive (h), represent (b_t), decide (π), act (a_t), adapt (learning).

Capability-oriented — what an autonomous system can do *without*:

Definition. A system is **autonomous** to the degree that it can pursue a goal in an environment **without requiring human intervention** to handle conditions that arise during execution.

The phrase "to the degree" matters. **Autonomy is a spectrum, not a binary.**

THREE NECESSARY PROPERTIES

Autonomy requires all three — they form a closed loop.

PROPERTY	WHAT IT MEANS	WITHOUT IT
Sensing	Perceive the environment	Any action is open-loop; cannot adapt
Deciding	Compute an action from information	This is where intelligence lives: the mapping $o \rightarrow a$
Acting	Change the world (or own state)	An observer only — not an autonomous system

THE SAE AUTONOMY SPECTRUM

Society of Automotive Engineers (SAE) [J3016](#) (2014, revised 2021) — written for road vehicles, but the logic applies to any autonomous system: *who monitors, who intervenes, and under what conditions?*

LEVEL	LABEL	HUMAN ROLE	EXAMPLE
0	Manual	Full control	Bicycle, old car
1	Assisted	Human supervises; machine executes one function	Cruise control
2	Partial	Machine: speed + lane; human monitors	Tesla Autopilot
3	Conditional	Machine drives; human ready to take over on request	Waymo early pilots
4	High	All conditions within a defined operational domain	Waymo Rider-Only
5	Full	All conditions everywhere; no steering wheel needed	Not yet deployed

TESLA FSD – FULL SELF-DRIVING IN ACTION

<div style="text-align:center; margin-top: 2em;">

[WATCH ON YOUTUBE](#)

youtu.be/Z6qp8MLfVLA

</div>

**TESLA FSD MARKETS IT AS "FULL SELF-DRIVING." WHAT
SAE LEVEL IS IT – AND WHY?**



ANSWER

SAE Level 2 — despite the name.

At **Level 3**, the machine drives and the human only needs to respond when the machine explicitly asks for help. Responsibility transfers to the machine.

At **Level 2**, the driver must remain alert and ready to intervene **at any moment** — responsibility never transfers, regardless of how much the car is doing.

FSD never transfers responsibility. The human is always the fallback. That is the L2/L3 boundary — and it is a legal and ethical line, not just a technical one.

THE OPERATIONAL DESIGN DOMAIN (ODD)

The key concept separating Level 4 from Level 5.

ODD — the specific set of conditions under which a given system is designed to operate: road type, geography, weather range, speed range, time of day, and so on.

- Every **Level 1–4** system has an ODD
- **Level 5**, by definition, does not
- Outside its ODD, a Level 4 system must either **stop safely** or **hand control to a human**

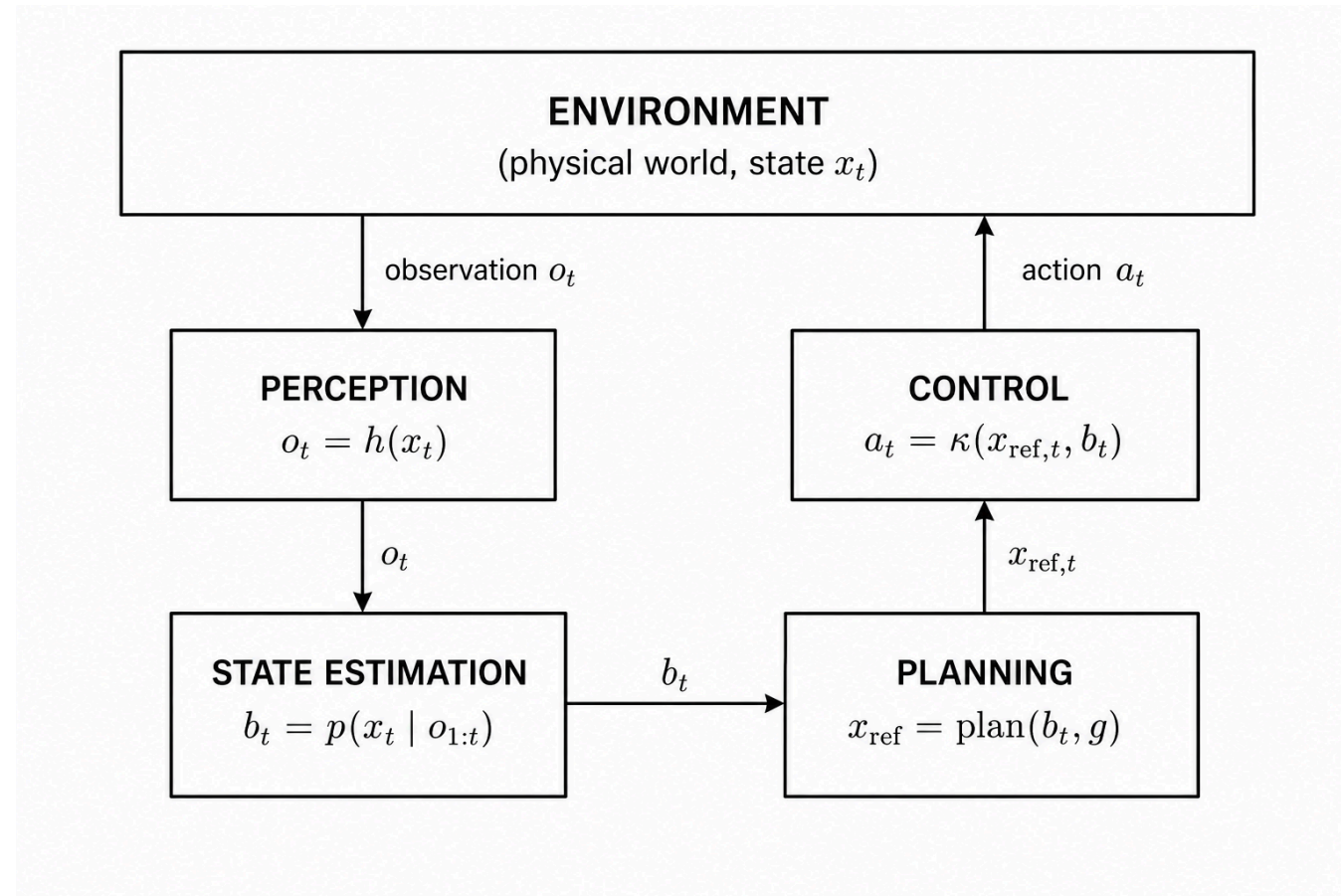
What changes moving up the spectrum? The machine must increasingly handle: unexpected events, partial information, long horizons, and diverse environments. All of these translate into precise mathematical properties of the tuple.

§2 · THE PERCEPTION-DECISION-ACTION LOOP

THE FUNDAMENTAL ARCHITECTURE OF EVERY AUTONOMOUS SYSTEM



THE FOUR-STAGE LOOP



Each stage runs at a **potentially different frequency**. Signals at each interface are named explicitly.

**THE LOOP HAS FOUR STAGES. WHICH ONE MUST RUN
FASTEST – AND WHY DOES PHYSICS DEMAND IT?**



ANSWER

Control — at up to **1 kHz**.

Physical systems have fast, potentially unstable dynamics. A drone's attitude can diverge in milliseconds if uncorrected — the inner loop must run faster than the physics can go wrong.

Planning can run at 1–10 Hz because it is asking *where to go*, not *how to stay in the air*. It only needs to react to significant changes in the belief or goal, not to every sensor tick.

The hierarchy exists because **different questions operate on different timescales** — not because of arbitrary engineering convention.

STAGE 1 – PERCEPTION

$$o_t = h(x_t)$$

The observation is:

- **Incomplete** — h is not invertible; o_t does not fully determine x_t
- **Noisy** — real sensors add measurement noise on top of the structural information loss
- **High-dimensional** — a camera gives megapixels; a LiDAR gives millions of points per second

The job of perception: convert o_t into a form suitable for state estimation.

STAGE 2 – STATE ESTIMATION

Since o_t alone is insufficient, the system maintains a **belief** — a probability distribution over all states consistent with observations so far:

$$b_t = p(x_t \mid o_{1:t}, a_{0:t-1})$$

- b_t is a *distribution*, not a point — it carries uncertainty
- b_t is a *sufficient statistic* for the full history $o_{1:t}$

The Bayesian belief is the *conceptually complete* answer. Practical estimators are approximations to it.

ESTIMATOR ZOO

APPROACH	WHAT IS MAINTAINED	BAYESIAN?
Kalman filter	Gaussian (μ_t, Σ_t)	Yes — exact for linear Gaussian
Particle filter	Weighted sample set $\{x^i, w^i\}$	Yes — approximate
Maximum A Posteriori (MAP) / point estimate	Single $\hat{x}_t$	No — discards uncertainty
Recurrent network	Hidden vector h_t	Not explicitly
No estimator	Raw o_t direct to planner	No

THE BAYES FILTER

The belief updates through two alternating steps:

$$b_t(x_t) \propto \underbrace{p(o_t \mid x_t)}_{\text{measurement update}} \cdot \underbrace{\int p(x_t \mid x_{t-1}, a_{t-1}) b_{t-1}(x_{t-1}) dx_{t-1}}_{\text{prediction step}}$$

Prediction step — propagate belief through the dynamics: where could the system be now, given where it was and what it did?

Measurement update — reweight by how well each candidate state explains the new observation.

Two things to notice:

1. b_t is a *distribution*, not a point — it carries uncertainty
2. b_t is a *sufficient statistic* for the full history $o_{1:t}$ — the planner needs only b_t

**WHY MAINTAIN A FULL DISTRIBUTION RATHER THAN
JUST THE BEST ESTIMATE?**



ANSWER

Because **uncertainty itself is decision-relevant**.

If you are 95% certain of your position, you act confidently. If you are 50% certain, you should act more cautiously — slow down, gather more data, avoid the irreversible action.

A point estimate $\hat{x}_t$ collapses the distribution and discards exactly that information. The planner can no longer distinguish "I know where I am" from "I have no idea where I am" — both look the same.

b_t is not just a best guess. It is everything the system knowably knows about x_t , given all observations so far.

STAGE 3 – PLANNING

Given belief b_t and goal specification g , the planner decides *what the system should do at a high level*:

$$x_{\text{ref}} = \text{plan}(b_t, g)$$

g tells the planner *what counts as success* — it does not prescribe the sequence of actions. g is typically **fixed or slowly changing** (from an operator or language model), while b_t updates at sensor rate.

Planning is the **slowest** stage in the loop: **1–10 Hz**. It reasons about *where to go*, not *how to get there* precisely — that is the controller's job.

GOAL SPECIFICATION

The form of g changes as we move through the course:

MODULE	FORM OF G	EXAMPLE
IV — Classical control	Target state $x^* \in \mathcal{X}$	"Reach position $(3, 5)$ "
IV — Optimal control	Cost functional $\int_0^T l(x_t, a_t) dt + \phi(x_T)$	"Minimise fuel while reaching x^* "
V — RL	Reward function $r : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$	"Maximise cumulative score"
VI — Foundation models	Natural language instruction	"Pick up the red cup"

The common thread: g defines success. It is an exogenous input, not an internal state of the system.

STAGE 4 – CONTROL

Given reference x_{ref} and belief b_t , the controller computes the **low-level action** sent to the actuators:

$$a_t = \kappa(x_{\text{ref}}, b_t)$$

The controller's job is *execution*: track the plan as closely as possible despite disturbances, modelling errors, and actuator limits. It runs at **high frequency** — a drone attitude controller runs at **1 kHz**, orders of magnitude faster than the planner.

Classical controllers: **PID, LQR, MPC** (Lectures 7–8).

PLANNING VS. CONTROL – THE HIERARCHY

LAYER	QUESTION	HORIZON	FREQUENCY	METHOD
Planning	<i>Where to go?</i>	Long (T steps)	Low (1–10 Hz)	Search, OCP, RL
Control	<i>How to get there?</i>	Short (1–few steps)	High (100 Hz–1 kHz)	PID, LQR, MPC

The decomposition is not always clean — MPC blurs the boundary by solving a short-horizon planning problem at control frequency. Learned end-to-end policies blur it further. But the conceptual separation is useful because **the failure modes are different**: a bad plan leads the system to the wrong place; a bad controller fails to execute a good plan.

LEARNED POLICIES IN THE LOOP

The common thread: a classical fast loop persists underneath every learned component.

Pattern 1 — RL in the planning layer (*legged locomotion, manipulation*)

RL policy at ~25 Hz outputs desired joint positions q^{ref} ; a PD controller at ~1 kHz converts to torques:

$$\tau = K_p(q^{\text{ref}} - q) + K_d(\dot{q}^{\text{ref}} - \dot{q})$$

LEARNED POLICIES – PATTERNS 2 & 3

Pattern 2 — RL replaces the outer-loop controller (UAVs)

RL takes the outer position/velocity loop; the onboard PID rate controller remains at high bandwidth. RL learns *where to point and how fast*; PID ensures the attitude gets there.

Pattern 3 — RL tunes PID gains (industrial process control)

RL observes tracking error and adapts gains (K_p, K_i, K_d) online. The PID structure is kept intact; RL is a meta-controller, not a replacement.

What changes is *which layer is learned*, not whether the hierarchy exists.

WHY FEEDBACK MATTERS

The loop closes. The action affects the state, which affects the next observation, which affects the next belief, plan, and action.

BEHAVIOUR	
Open-loop	Pre-planned action sequence; no correction for what actually happens
Closed-loop	Continuously corrects based on observations

Almost all non-trivial autonomous behaviour requires the loop to be **closed**.

MULTI-RATE LOOP

Each layer runs at its own frequency. Example — a drone:

LAYER	FREQUENCY
Task planner	~1 Hz
Trajectory planner	~10 Hz
Position control	~100 Hz
Attitude control	~1 kHz

The mathematical framework must account for discretisation at every layer.

§3 · EXAMPLES — THE PDA LOOP IN PRACTICE

THREE SYSTEMS WE RETURN TO THROUGHOUT THE COURSE



EXAMPLE 1 – WAYMO ONE (SAE LEVEL 4)

Robotaxi operating without a safety driver in Phoenix and San Francisco. Handles all driving tasks within a defined geo-fenced ODD.

STAGE	CONCRETE REALISATION	SIGNAL
Perception	Cameras, LiDAR, radar; neural nets detect vehicles, pedestrians, traffic lights	o_t
State estimation	Multi-sensor Kalman filter; 5 cm-accurate localisation; tracks all nearby agents	b_t
Planning	Behaviour planner (lane changes, turns) → trajectory planner at ~10 Hz	x_{ref}
Control	Feedback controller: steering, throttle, brake at ~100 Hz	a_t

Why Level 4 and not 5? The geo-fence. ODD is finite and carefully mapped.

The gap from Level 2 to Level 4 is not better sensors alone — it is engineering every link in the loop to **degrade gracefully** rather than fail catastrophically.

EXAMPLE 1 – WAYMO ENVIRONMENTAL ENVELOPE

CONDITION	CAN HANDLE?
Rain, moderate fog, darkness	✓ Within ODD
Unexpected pedestrians, construction zones	✓ Within ODD
Double-parked vehicles, novel traffic patterns	✓ Within ODD
Heavy snowfall (sensor occlusion)	✗ Exceeds design tolerance
Roads not in the high-definition map	✗ Outside geo-fence
Flooding, conditions outside geo-fence	✗ Car stops, calls for human

The ODD boundary is the exact line between Level 4 and Level 0 — outside it, the car stops and calls for a human.

EXAMPLE 2 – QUADROTOR DRONE

GPS-guided quadrotor flying a pre-planned inspection mission over a construction site.

STAGE	CONCRETE REALISATION	SIGNAL
Perception	IMU, cameras, GPS, barometer	o_t
State estimation	EKF fuses IMU + GPS; estimates position, velocity, attitude	b_t
Planning	Waypoint trajectory from mission file; replanned on obstacle detection	x_{ref}
Control	Two nested loops: position at ~100 Hz, attitude at ~1 kHz	a_t

KEY SENSORS – IMU AND INS

IMU (Inertial Measurement Unit) — accelerometers + gyroscopes. Self-contained, no external signals. Limitation: small errors accumulate when integrated → position drift over time.

INS (Inertial Navigation System) — IMU plus integration algorithms that produce **position, velocity, and attitude** estimates. The accumulated drift is the INS's fundamental limitation — GPS (or another absolute reference) periodically corrects it.

EXAMPLE 2 – DRONE AUTONOMY IS CONTEXT-DEPENDENT

Autonomy level is not a property of the system — it is a property of the system in an environment.

ENVIRONMENT	SAE EQUIVALENT	WHAT IT CANNOT HANDLE
Open airspace, reliable GPS, mild wind	Level 3–4	GPS jamming, sensor failure
Urban canopy, weak GPS signal	Level 2	Needs human oversight for positioning
GPS-denied (indoor, tunnel)	Level 1	Human must provide position reference
Dense obstacles, no active sensing	Level 1	No awareness of what it will hit

EXAMPLE 3 – ROBOTIC ARM (WAREHOUSE MANIPULATION)

Scanning barcodes, planning grasps, sorting packages.

STAGE	CONCRETE REALISATION	SIGNAL
Perception	RGB-D camera; object detection and 6-DoF pose estimation network	o_t
State estimation	Joint angles from encoders (known exactly); <i>object pose</i> maintained as a distribution	b_t
Planning	Grasp planner → collision-free joint trajectory	x_{ref}
Control	PD or impedance controller at ~1 kHz	a_t

Key insight: the belief is not only about the robot's own state. The distribution over object pose is uncertain even when joint angles are exactly known.

Most industrial manipulation today is **Level 2 equivalent** — the robot executes fine motion reliably, but a human must set up the environment and handle failures.

EXAMPLE 3 – ARM ENVIRONMENTAL ENVELOPE (LEVEL 2)

CONDITION	CAN HANDLE?
Small position variation of known objects	✓ Within grasp planner tolerance
Normal factory lighting, clean conveyor flow	✓ Known object geometries
Novel object shapes not in the grasp library	✗ Grasp planner fails
Significant clutter or occlusion	✗ Breaks vision pipeline
Lighting changes	✗ Breaks vision pipeline
Unstructured environment (kitchen, disaster site)	✗ Effectively Level 0

The Level 2 rating is achieved not by making the arm smarter but by **engineering the environment to be predictable**.
The arm's ODD is the controlled factory floor itself.

EXAMPLE 4 – SHAHED-136

Delta-wing aircraft, 200 kg, 50 kg warhead, ~2 500 km range. Once launched: flies to a GPS coordinate and detonates. No pilot, no uplink during flight.

STAGE	CONCRETE REALISATION	SIGNAL
Perception	GPS/GLONASS + INS (barometer, compass, accelerometers) — no environment sensing	o_t
State estimation	GPS-corrected INS; falls back to dead-reckoning INS when GPS is jammed	b_t
Planning	Waypoint sequence uploaded before launch; no replanning	x_{ref}
Control	Autopilot tracks waypoint trajectory at cruise altitude	a_t

What is not in the loop: no obstacle avoidance, no situational awareness, no target-seeking sensor in its basic form.

EXAMPLE 4 – SHAHED: HIGH AUTONOMY, MINIMAL PERCEPTION

High autonomy level does not require sophisticated perception.

Once launched: zero human intervention to reach and strike a target — by the capability definition, **Level 4–5**. But the ODD is extraordinarily narrow: *fly these coordinates in uncontested airspace*.

CONDITION	CAN HANDLE?
Headwinds / tailwinds within autopilot authority	✓
Minor GPS signal degradation (multi-constellation)	✓
Active GPS jamming	✗ Falls back to degrading INS dead-reckoning
Kinetic interception	✗ No evasion capability
Any obstacle in the flight path	✗ No obstacle awareness
Target coordinate errors	✗ Hits wherever told, not where the target is

The high autonomy level and the minimal perception are the **same design choice**: achieve Level 4–5 by accepting an environment in which sophisticated perception is not needed.

GPS-JAMMING FALLBACK – A STATE ESTIMATION LESSON

GPS + INS is a classic **sensor fusion** problem:

- **GPS** provides corrected position measurements (resets growing INS error at each fix)
- **INS** provides dead-reckoning that drifts over time without correction

When GPS is jammed and the fix disappears:

b_t becomes progressively wider — uncertainty grows

The system keeps flying because it has **no mechanism to stop or report failure**. The belief degrades **silently**.

**THE SHAHED KEEPS FLYING WHEN GPS IS JAMMED AND
ITS BELIEF DEGRADES SILENTLY. WHAT SHOULD A WELL-
DESIGNED SYSTEM DO INSTEAD?**



ANSWER

Monitor its own uncertainty — and act on it.

When the belief over position b_t becomes too wide, the system no longer knows where it is reliably. A well-designed system:

1. **Detects** the degrading belief (uncertainty exceeds a threshold)
2. **Triggers a safe behaviour** — loiter, return to last known position, stop
3. **Reports the failure explicitly** rather than proceeding on a collapsing estimate

The Shahed has none of this. It has no mechanism to represent uncertainty, no threshold to detect degradation, and no safe fallback. It keeps flying because it cannot know that it is lost.

Knowing what you don't know is part of autonomous intelligence.

EXAMPLE 5 – BALLISTIC MISSILE (THREE-PHASE LOOP)

A ballistic missile has **qualitatively different PDA loop behaviour** in each phase.

PHASE	PERCEPTION	ESTIMATION	PLANNING	CONTROL
Boost	IMU	INS integration	Pre-computed trajectory	Thrust vectoring
Midcourse	None	None (open loop)	None	None
Terminal	INS $\pm$ GPS $\pm$ seeker	Position + target state	Intercept geometry	Fins / attitude thrusters

Midcourse phase — the motor is off; the missile is a projectile on a ballistic arc. There is no perception, estimation, or control. This is not a limitation — it is a deliberate design choice. Near-vacuum and Newtonian gravity are completely predictable; no loop is needed.

S4 · HISTORICAL ARC

FIVE ERAS – EACH SOLVED ONE PROBLEM AND EXPOSED THE NEXT



ERA 1 – PERFECT MODEL, PERFECT PLAN (1950S-1980S)

Founding assumption: intelligent behaviour is a reasoning problem. Give a machine an accurate model and correct inference rules; behaviour follows.

SHAKY (SRI, 1966–1972) — wheeled robot with camera and bump sensors; planner **STRIPS** searched through a symbolic state space.

Closed-world assumption: anything not stated explicitly is assumed false.

ERA 1 – THE LIMITS

The environment had to be engineered to match the model:

- Objects in known positions
- Sensing reliable enough to maintain an exact world state
- Any unexpected change — a block moved, a shadow shifted — caused **unrecoverable plan failure**

In a real warehouse, on a real road, the closed-world assumption is violated constantly.

The lesson: perfect models do not exist. The question is what to do with imperfect ones.

ERA 2 – THROW OUT THE MODEL (1986-1995)

Rodney Brooks (MIT): the entire deliberate–plan–act architecture was wrong.

Behaviour-based robotics — layer simple reactive rules, no central world model.

Genghis (six-legged walker) navigated office hallways through pure local reaction, coordinated by physical interaction — not a planner. Robust in ways SHAKEY never was.

**A REACTIVE ROBOT NAVIGATES AN OFFICE HALLWAY
PERFECTLY WITHOUT A MAP. CAN IT PLAN A ROUTE
ACROSS CAMPUS? WHY NOT?**



ERA 2 – THE CEILING

Reactive systems cannot pursue long-horizon goals. Genghis could not:

- Plan a route across a city
- Remember where it had been
- Reason about future states

Reaction is robust. Purpose requires memory and prediction.

The real lesson: models must handle *incompleteness and uncertainty* — not pretend to be complete.

ERA 3 – EMBRACE UNCERTAINTY (1990S-2005)

Sebastian Thrun, Wolfram Burgard, Dieter Fox: import probability theory wholesale into robotics.

The key move: the robot does not *know* its state — it maintains a *belief* b_t , a probability distribution updated with every new observation. Uncertainty is not a failure to be fixed; it is a **first-class citizen** in the formalism.

Tools: Kalman filters and particle filters, applied to **localisation** and **SLAM** (simultaneous localisation and mapping). For the first time, a robot could build a map of an unknown environment while navigating it using only onboard sensors.

DARPA GRAND CHALLENGE (2005)

Proof of concept: five vehicles completed 132 miles of unpaved desert roads autonomously.

Stanford's **Stanley** — EKF localisation, probabilistic road model, PID controller. No learning, no neural networks. Just careful probabilistic engineering.

Bottleneck immediately exposed: perception.

The estimator worked beautifully given clean features. The handcrafted feature extractors that converted raw sensor data into those features were fragile — designed for specific conditions, broken outside them.

The probabilistic framework was correct. The interface to the physical world was the weak link.

ERA 4 – LEARN TO SEE, LEARN TO ACT (2010S)

2012 ImageNet: AlexNet cut top-5 error from 26% to 15% in one year. Within five years, every perception pipeline in robotics was a neural network. The handcrafted h replaced by a **learned** one.

Deep RL extended this to π :

- **DQN** (DeepMind, 2013) — Atari at superhuman level from raw pixels
- **AlphaGo** (2016) — defeated the world Go champion

ERA 4 – THE HARD CONSTRAINT

Both h and π can be learned. But at what cost?

- Learning Atari required **tens of millions** of game frames
- Physical robots break — cannot run millions of real-world experiments
- **Sim-to-real gap:** subtle mismatches in friction, latency, and visual appearance caused policies trained in simulation to fail in reality

The old generalisation problem had not disappeared — it had moved from perception into policy.

ERA 5 – GENERALISATION AT SCALE (2020-PRESENT)

The **transformer** architecture, trained on internet-scale text, images, and video, learned representations that transferred across tasks in unprecedented ways.

When applied to robotics — training on robot demonstrations combined with internet data:

RT-2 (Google, 2023) — robot arm following instructions like “*pick up the extinct animal*” with no training on that specific phrasing or object, drawing on visual and semantic knowledge from web-scale pretraining.

Current frontier: Vision-Language-Action models (VLAs), diffusion policies, world models.

Working hypothesis: at sufficient scale, h and π can be learned jointly in ways that generalise across tasks — removing the need for handcrafted pipelines.

Open questions: reliability far below safety-critical demands; enormous compute and data; interpretability nearly absent.

THE THROUGH-LINE

Each era solved one problem and immediately exposed the next:

ERA	KEY INSIGHT	KEY FAILURE IT EXPOSED
Deliberative (1960s–80s)	Planning is tractable with a perfect model	Models are never perfect
Reactive (1986–95)	Local reactions are robust without a model	Long-horizon goals require memory and prediction
Probabilistic (1990s–2005)	Uncertainty is tractable; maintain a belief	Perception is the bottleneck
Deep learning (2010s)	h and π can be learned	Sample efficiency and generalisation
Foundation models (2020s)	Generalisation across tasks is achievable at scale	Reliability, safety, interpretability

WHAT'S NEXT

Lecture 2 — Dynamical Systems

Precise definitions of $\mathcal{X}, \mathcal{O}, \mathcal{A}, f, h$ · Stability · Linearisation · Discretisation